

合数模数下的区间生成子模成员查询

带时间标记的 p -进阶梯表及其 Lean 4 验证

ASTRA4OI 项目

2026 年 9 月 16 日 技术论文稿

摘要

给定向量序列 $a_i \in (\mathbb{Z}/m\mathbb{Z})^d$ ，区间生成子模成员查询判断目标是否属于 $\langle a_l, \dots, a_r \rangle$ 。合数模数下的非单位主元具有非平凡零化关系，直接沿用域上的单主元消元不能保证完备性。本文给出带时间标记的 p -进阶梯表算法：对模 p^k 的每个新向量只启动至多 k 条不分叉幂链，按时间优先处理被替换的余行。证明利用循环商群的幂层首项与数字表示基数，建立同时适用于所有后缀的不变量。正维度下，预处理主体坐标更新量为 $O(nkd^2)$ ，单次成员查询为 $O(d^2)$ ；已知素因子分解时，通过中国剩余定理合并为 $O(nKd^2)$ 和 $O(wd^2)$ 。我们给出一般性 Lean 4 证明、逐条定理对照与独立参考程序测试。所述界将求逆、赋值、堆操作及整数位成本明确单列，既不假定因数分解免费，也不声称形式化的关系式定义等同于经过源码精化验证的可执行实现。

关键词：合数模线性代数；区间查询；有限链环；数字基；形式化验证

1 引言与问题定位

域上线性基允许通过可逆主元逐列消元，而 $\mathbb{Z}/m\mathbb{Z}$ 中的非零元素不一定可逆。例如模 4 下， $(2, 1)$ 的倍数 $(0, 2)$ 不能通过只保留第一列主元的朴素结构识别。此外，序列 $1, 2 \pmod{4}$ 的整个区间可以生成 1，最后一个元素却不能；因此，仅保留最新原向量也不足以支持后缀过滤。

本文研究追加型序列上的成员判断，不要求输出系数、不支持中间元素修改或删除。核心方法在每列保留 k 个赋值槽位，并用时间戳同时表示所有后缀。主要交付是：一个完整的插入与查询算法；基于循环扩张和不分叉链的证明；以及连接论文编号、Lean 声明和可复现实验的研究工件。

相关工作与范围。 单个区间的模成员关系可以归约为整数格成员关系。Smith 与 Hermite 正规形的多项式算法由 Kannan 和 Bachem 给出 [2]；这些正规形提供了静态问题的基础方法。有限环上限制系数为数字集合的基并非新的概念，例如 Kuijper 和 Pinto 在环卷积码中使用 p -线性组合与 p -基 [3]。本文采用适合有限向量和首项槽位的明确定义，关注所有后缀的时间标记维护，而不将一般数字基概念本身作为创新性主张。形式化使用 Lean 4 [1] 与 Mathlib [4]。本文不主张所得界具有已知最优性，也不以实验时间与正规形算法作未经控制的性能比较。

2 记号、接口与成本模型

令 $m = \prod_{s=1}^w p_s^{k_s} > 1$, 其中 p_s 为互异素数, $k_s \geq 1$, 并记 $K = \sum_s k_s$ 。分解作为已知输入。先固定素数幂环 $R = \mathbb{Z}/p^k\mathbb{Z}$, 其中 $n, d, k \geq 1$ 。坐标编号为 $0, \dots, d-1$, 原向量时间编号为 $1, \dots, n$ 。

定义 1 (区间子模). 对 $1 \leq l \leq r \leq n$, 定义

$$M_{l,r} = \text{span}_R\{a_i : l \leq i \leq r\}.$$

成员查询要求返回布尔值 YES 当且仅当 $x \in M_{l,r}$ 。

Lean: Paper.intervalSpan; Paper.prefixSource_span.

一个坐标更新指归一化时的一次坐标缩放, 或行消元时的一次乘减更新, 它可包含常数标量模运算。我们将其与单位求逆次数、赋值次数、堆比较和输入扫描分别计量。槽位采用可常数时间寻址的数组, 优先队列采用大小相关的对数成本堆。后文的精确常数界针对坐标更新额度; 完整标量成本和位成本另行说明。

定义 2 (首项与规范槽位). 对非零 $b \in R^d$, 令 j 为首个非零坐标, $v = v_p(b_j) \in \{0, \dots, k-1\}$, 并定义 $\text{lead}(b) = (j, v)$, 按字典序排序。槽位 (j, v) 的规范行满足 $b_i = 0$ ($i < j$) 且 $b_j = p^v$ 。

Lean: PadicEchelon.HasLead; PadicEchelon.PivotRow.

若首坐标的规范代表元是 $p^v u$, 则 u 为单位, 乘 u^{-1} 即可归一化。消元系数 b_j/p^v 是规范整数代表元上的整除, 不涉及非单位 p^v 的逆元。

3 数字基与循环商群

定义 3 (p -数字基). 首项互异的规范行族 $\mathcal{B}$ 是子模 H 的 p -数字基, 当且仅当映射

$$\{0, \dots, p-1\}^{\mathcal{B}} \longrightarrow H, \quad (c_b)_b \longmapsto \sum_{b \in \mathcal{B}} c_b b$$

为双射。这不要求 H 是自由 R -模。

Lean: DigitCardinality.CardinalBasis; DigitCardinality.CardinalBasis.mem_iff_digitRep.

引理 1 (数字表示单射). 任何首项互异的规范行族有恰好 $p^{|\mathcal{B}|}$ 个不同数字组合。

证明. 两个不同组合相减, 取非零系数对应的最小首项 (j, v) 。该系数非零且绝对值小于 p , 故不被 p 整除。在坐标 j , 其贡献有精确赋值 v ; 同列更晚项的赋值至少为 $v+1$, 而后列项在此坐标为零, 因此不能抵消。□

Lean: DigitCardinality.encode_injective; DigitCardinality.card_range_encode.

引理 2 (数字基查询完备性). 给定 H 的数字基, 非零成员的首项一定对应其中的槽位。按规范主元逐列消元, 遇到缺槽返回 NO、最终归零返回 YES, 所得答案正确, 且至多进行 d 次行消元。

证明. 数字展开的首个非零项决定整个向量的首项, 故成员不会在缺槽处停下。减去基行的任意环倍数保持成员关系。反向累加成功消元中的行倍数证明 YES 的充分性。每步清空整个当前坐标, 已经清空的前缀不再改变, 故消元次数至多为 d 。□

Lean: BasisQuery.exists_correct_answer;BasisQuery.Derivation.correct.

引理 3 (循环商群的幂层). 对 $H \leq R^d$ 与 $a \in R^d$, 存在 $0 \leq h \leq k$ 使得 $\text{ord}(a + H) = p^h$ 。而 $p^e a + H \neq 0$ 当且仅当 $e < h$ 。扩大 H 只能使 h 减小。

证明. R^d/H 被 p^k 零化, 故元素阶整除 p^k , 从而是某个 p^h 。 p^e 零化该陪集当且仅当 $p^h \mid p^e$ 。较大商模是较小商模的商, 元素阶只能取约数。□

Lean: CyclicQuotient.exists_quotient_order_exponent;CyclicQuotient.nsmul_pow_ne_zero_iff;CyclicQuotient.quotient_exponent_anti.

引理 4 (陪集的缺槽首项). 相对于 H 的数字基, 消元到首次缺槽得到的非零余行, 其首项仅由陪集决定。乘单位不改变该首项; 乘 p 并重新消元后, 首项严格增大, 或陪集归零。

证明. 若同陪集的两个缺槽余行首项不同, 其差属于 H , 却仍具有较早的缺失首项, 与引理 2 矛盾。单位缩放不改变首项和缺槽性质。乘 p 会提高当前坐标的赋值或清空该坐标, 后续消元只会清空更多坐标, 因此严格向后推进。□

Lean: BasisQuery.Basis.missing_coset_lead_unique;BasisQuery.Derivation.p_nsmul_final_strictly_later.

引理 5 (循环扩张的数字基). 若 $\text{ord}(a + H) = p^h$, 则从每个存活幂层取得一个规范缺槽余行 $b_e \equiv u_e p^e a \pmod{H}$ (u_e 为单位, $0 \leq e < h$), 这些行与 H 的数字基合起来构成 $H + Ra$ 的数字基。

证明. 由引理 4, 新增各层的首项严格递增, 且均不属于旧基槽位。若旧基有 s 行, 合并后的 $s+h$ 行产生 p^{s+h} 个不同数字组合。这些组合都位于扩张子模中, 而 $|H + Ra| = |H|p^h = p^{s+h}$, 故注入映射为满射。这也证明不同存活幂层不会争用同一缺槽。□

Lean: ArbitraryExtension.basisFromSettledLayers;CyclicExtension.no_same_slot_of_missing_layers.

4 带时间标记的算法

表 $B[j][v]$ 保存规范行、时间戳 t 和原向量幂层 e 。时间戳与幂层记录的是商群身份

$$b \equiv up^e a_t \pmod{H_{t+1}^{(r)}}, \quad H_{t+1}^{(r)} = \text{span}_R\{a_{t+1}, \dots, a_r\}, \quad u \in R^\times. \quad (1)$$

幂层 e 不等于槽位赋值 v 。减去较新行必须保留旧时间与旧幂层。

每次替换只接续一个旧余行, 不额外生成新主元的 p 倍。幂链首次建立后只进行原任务的消元或转移。这个无分叉条件对复杂度证明不可省略。

算法 1 查询当前前缀的后缀 $[l, r]$ **Require:** 当前前缀表 B , 左端点 l , 目标 $x \in R^d$

```

1: for  $j = 0, \dots, d-1$  do
2:   if  $x_j = 0$  then
3:     continue
4:   end if
5:    $v \leftarrow v_p(x_j), b \leftarrow B[j][v]$ 
6:   if  $b$  不存在或  $b.t < l$  then
7:     return NO
8:   end if
9:    $x \leftarrow x - (x_j/p^v)b$  ▷ 所有坐标模  $p^k$ 
10: end for
11: return YES

```

算法 2 追加时间为 r 的向量 a_r

```

1:  $Q \leftarrow \{(p^e a_r, r, e, 0) : 0 \leq e < k, p^e a_r \neq 0\}$ 
2: while  $Q \neq \emptyset$  do
3:   弹出时间最大的  $(x, t, e, s)$ , 同时间以较小幂层优先
4:   for  $j = s, \dots, d-1$  do
5:     if  $x_j = 0$  then
6:       continue
7:     end if
8:      $v \leftarrow v_p(x_j), b \leftarrow B[j][v]$ 
9:     if  $b$  不存在 then
10:      归一化  $x$ , 以  $(t, e)$  存入槽位; break
11:     else if  $b.t > t$  then
12:        $x \leftarrow x - (x_j/p^v)b$ 
13:     else if  $b.t < t$  then
14:       归一化  $x$ , 用  $(x, t, e)$  替换槽位
15:        $y \leftarrow b - (b_j/p^v)x$ 
16:       if  $y \neq 0$  then
17:         将  $(y, b.t, b.e, j+1)$  压回  $Q$ 
18:       end if
19:       break
20:     else
21:       不可能发生, 见引理 7
22:     end if
23:   end for
24: end while

```

5 插入不变量与完备性

固定待完成的前缀 r , 令 p^{h_t} 是 $a_t + H_{t+1}^{(r)}$ 的阶。称 (t, e) 在 $e < h_t$ 时为必要键。考虑表和待处理队列组成的活动集合, 维持:

- (I1) 表中槽位互异, 且规范行按首项有序;
- (I2) 活动键 (t, e) 不重复, 每条活动行满足式 (1);
- (I3) 每个必要键都出现在表或队列中。

引理 6 (终态提取). 若一个表满足上述三个条件且队列为空, 则对任意 l , 时间至少为 l 的行构成 $M_{l,r}$ 的数字基; 时间 t 恰保留层 $0, \dots, h_t - 1$ 。

证明. 对时间从大到小归纳。假定较新行已构成 $H = H_{t+1}^{(r)}$ 的数字基。时间 t 的行槽位不属于较新表, 且每个 $e < h_t$ 由 (I3) 必定存在。若存在 $e \geq h_t$ 的行, 式 (1) 使它属于 H , 却在 H 的缺槽处有非零首项, 与引理 2 矛盾。由 (I2) 必要键不重复, 再应用引理 5 即得该时间对应的扩张数字基。□

Lean: TerminalExtraction.drained_table_correct.

引理 7 (活动不变量与碰撞排除). 从一个正确前缀表追加新向量, 初始队列满足 (II)–(I3)。每个合法分支保持这些条件。弹出最大时间任务时, 不可能与同时间的存储行在同槽位碰撞。

证明. 追加使每个旧 H_{t+1} 扩大, 故旧必要层集合只会缩小; 新时间的全部 k 层最初进入队列, 省略的零行不可能具有必要键。因此初始 (I3) 成立。规范化只乘单位, 旧行只减较新行, 保持商群身份。每次操作仅移动键或丢弃零行, 必要层的非零商群身份排除了后者, 故 (I2)–(I3) 保持。槽位替换和空槽写入保持 (I1)。

设弹出任务时间为 t 。队列中无较新任务, 所以所有较新必要键已位于表中。将活动不变量限制到较新时间, 便可应用引理 6, 得到 H_{t+1} 的完整数字基。若同时间两行碰撞, 其槽位为较新表的缺槽; 它们的键不同, 所以幂层不同。非零缺槽余行不可能来自已消失的层, 而不同存活层由引理 5 具有不同首项, 矛盾。该论证先从局部条件提取较新数字基, 不以插入已经成功完成为前提。□

Lean: FastAppend.initial_protected;FastInvariant.scan_preservesKeysAndProtected;PrioritySafety.same_time_stamp_collision_impossible.

引理 8 (终止与不分叉预算). 一次插入始终存在合法后继并在有限步内完成; 列访问至多 kd , 弹出至多 $k(d+1)$, 任一时刻待处理任务数至多为 k 。

证明. 非零行必有首项且可以单位归一化, 引理 7 排除了唯一的碰撞错误分支。每个初始任务拥有 d 的列预算。完成任务消耗一个队列元素; 替换只产生一个从下一列开始的后继, 并至少消耗一列。因此队列长度不增长, 总列访问不超过初始预算 kd 。势函数 $|Q| + \sum_{q \in Q} (d - q.\text{start})$ 每次弹出严格减小, 初值至多 $k(d+1)$, 同时给出终止性和弹出次数界。□

Lean: Totality.appendResult_nonempty;WorklistComplexity.insertion_visits_le;WorklistComplexity.insertion_pops_le.

6 主结果与完整成本说明

定理 1 (素数幂区间成员查询). 对任意素数 p 、正整数 k, d 和输入序列, 算法 2 从空表依次处理前缀, 在每个右端点 r 得到同时适用于全部 $[l, r]$ 的数字基表。算法 1 返回 YES 当且仅当 $x \in M_{l,r}$, 至多进行 d 次行消元。每个表至多有 dk 行; 处理 n 个向量的消元与初始幂链准备额度至多为 $3nkd^2$ 个坐标更新。

证明. 空表满足不变量。引理 7、8 保证每次插入合法终止, 引理 6 恢复所有后缀数字基。对前缀归纳, 再由引理 2 得查询结论。槽位总数为 dk 。每次列访问至多包含两次向量遍历, 因此消元额度至多 $2kd^2$; 一次输入规范化及至多 $k-1$ 次逐行乘 p 共预留 kd , 在 $d \geq 1$ 时总计不超过 $3kd^2$ 。□

Lean: Paper.theorem_1_prime_power.

推论 1 (子模大小). 若筛选后的表有 s 行, 则 $|M_{l,r}| = p^s$ 。

证明. 这是数字基定义中的双射及引理 1 的直接推论。□

Lean: TimestampedBasis.CorrectTable.card_at;Paper.prefixSource_span.

定理 2 (合数模数的 CRT 合并). 给定互异素数的分解 $m = \prod_{s=1}^w p_s^{k_s}$, 整体查询的答案为各分量答案的合取。其主体预处理坐标更新量为 $O(nKd^2)$, 每次成员查询为 $O(wd^2)$ 。

证明. 整体线性表示投影到各分量给出必要性。反之, 对每个原向量下标 i , 将各分量的系数 $\lambda_i^{(s)}$ 分别用 CRT 合成同一模 m 的系数 λ_i ; 不要求各分量首先得到相同的整数系数。计数对分量求和, 分别得到 $3nKd^2$ 与 wd^2 。□

Lean: Paper.theorem_2_composite_interval;Paper.theorem_2_composite_preprocessing.

命题 1 (实现成本模型). 采用数组槽位和二叉堆, 预存各次 p 幂, 以二分整除性测试计算赋值。若除求逆外的标量算术暂按单位成本计, 一次单位求逆的成本为 $I(p^k)$, 则 n 次追加与 q 次查询的总上界为

$$O\left(nkd(d + \log(k+1) + I(p^k)) + qd(d + \log(k+1))\right). \quad (2)$$

证明. 引理 8 给出 $O(kd)$ 次堆操作; 每次成本 $O(\log(k+1))$ 。插入的赋值和求逆分别至多 kd 次, 查询赋值至多 d 次且不求逆。每次弹出至多 $O(d)$ 的零检测和坐标搜索亦被主体界吸收。预存幂的 $O(k)$ 构造成本在正 n, d 下被吸收。对查询按右端点分桶的 $O(n+q)$ 开销同理。□

该命题的事件次数由 Lean 验证; 数组、堆和标量子程序的单位成本契约是文字分析部分。任意精度整数需要逐类计入位成本, 因数分解成本另加。 $m = 1$ 时所有合法查询为 YES, 但输入输出和管理开销仍非零。程序的 span_size 需 $O(dK)$ 次槽位检查及额外幂运算, 不适用成员查询的界。

存储与历史区间。 当前表使用 $O(Kd^2)$ 个坐标单元。每次插入至多创建 $O(Kd)$ 个不可变行记录，保存每个前缀的 dK 个引用快照得到 $O(nKd^2)$ 空间，查询直接访问版本 r 。这是持久化布局的计数分析，不是分配器或字节级内存的形式化验证。

7 Lean 4 对照与信任边界

形式化将向量表示为 $\text{Fin } d$ 到 $\text{ZMod}(p^k)$ 的函数，以规范首项、生成子模和基数刻画数字基。`Paper.prefixSource` 将原序列前 r 项标为 $1, \dots, r$ ；`Paper.prefixSource_span` 证明时间阈值过滤等于定义 1 的字面区间子模。因此定理 1 的机器声明并不只针对抽象的“假定正确表”。

构造层的 `BuildResult` 证明字段由归纳实际建立；终止性和查询存在性均有证明。`noncomputable` 与选择公理用于选取数学证据，不意味着这些 Lean 定义编译后具有数组/堆程序的运行时间。同时时间幂层递增是参考程序的确定性次序；核心证明只要求弹出最大时间，因而覆盖此特例。

下表省略公共命名空间 `CompositeModulusBasis`。表 1 由 `paper/theorem-map.json` 生成。其中每个声明都接受编译检查和传递公理审计；允许的逻辑基础仅为 `propext`、`Classical.choice`、`Quot.sound`。有限模型回归被单列，不导入核心入口。Python 源码精化、通用有限环归约和完整位复杂度不在核心机器定理中。

表 1: 论文与 Lean 声明对照。

编号	结论	Lean 声明
定义 1	区间子模与下标桥接	<code>Paper.intervalSpan</code> <code>Paper.prefixSource_span</code> <code>Paper.compositePrefix_span</code>
定义 2	首项和规范行	<code>PadicEchelon.HasLead</code> <code>PadicEchelon.PivotRow</code>
定义 3	数字基的基数刻画	<code>DigitCardinality.CardinalBasis</code> <code>DigitCardinality.CardinalBasis.mem_iff_digitRep</code>
引理 1	数字表示单射	<code>DigitCardinality.encode_injective</code> <code>DigitCardinality.card_range_encode</code>
引理 2	查询存在性与双向正确性	<code>BasisQuery.exists_correct_answer</code> <code>BasisQuery.Derivation.correct</code>
引理 3	商群阶与幂层	<code>CyclicQuotient.exists_quotient_order_exponent</code> <code>CyclicQuotient.nsmul_pow_ne_zero_iff</code> <code>CyclicQuotient.quotient_exponent_anti</code>

编号	结论	Lean 声明
引理 4	缺槽首项及乘 p 的推进	<code>BasisQuery.Basis.missing_coset_lead_unique</code> <code>BasisQuery.Derivation.p_nsmul_final_strictly_later</code>
引理 5	循环扩张与层间不碰撞	<code>ArbitraryExtension.basisFromSettledLayers</code> <code>CyclicExtension.no_same_slot_of_missing_layers</code>
引理 6	终态提取后缀数字基	<code>TerminalExtraction.drained_table_correct</code>
引理 7	必要键保持和同时间碰撞排除	<code>FastAppend.initial_protected</code> <code>FastInvariant.scan_preservesKeysAndProtected</code> <code>PrioritySafety.same_timestamp_collision_impossible</code>
引理 8	合法终止、列预算与队列大小	<code>Totality.appendResult_nonempty</code> <code>WorklistComplexity.insertion_visits_le</code> <code>WorklistComplexity.insertion_pops_le</code> <code>WorklistComplexity.insertion_queue_lengths_le</code>
定理 1	素数幂区间主定理	<code>Paper.theorem_1_prime_power</code>
推论 1	区间子模大小	<code>TimestampedBasis.CorrectTable.card_at</code> <code>Paper.prefixSource_span</code>
定理 2	CRT 区间查询及预处理	<code>Paper.theorem_2_composite_interval</code> <code>Paper.theorem_2_composite_preprocessing</code>
命题 1	完整实现成本模型（仅事件计数）	<code>VerifiedScan.Drain.initial_operational_bounds</code> <code>WorklistComplexity.insertion_heap_comparisons_le</code>

8 实验设计与可复现性

参考实现只使用 Python 标准库。测试分为三种独立答案来源：小模数直接枚举所有系数；大模数使用整数格 $\text{span}_{\mathbb{Z}}(a_l, \dots, a_r, mI_d)$ 的三角化与整数回代；以及已知可逆坐标变换下的 gcd 理想构造和一维 gcd 判定。整数格 oracle 先通过 4,656 个小模数目标验证，并另与 9,484 个大模数构造目标对照。

表 2: 程序验证的覆盖与统计。成员断言包含三种接口/阶段的重复核对。

测试组	序列	区间	成员关系断言
小模数主测试	9,278	79,694	5,959,998
大模数新增测试	78	9,332	240,420

大参数使用种子 20260915, 质数独立试除验证, 最大为 2,147,483,647。包含 2^{64} 、 3^{20} 、 $(10^9 + 7)^3$, 以及 $998244353^2 \cdot 1000000007^2$ 的 120 位模数。维度为 1, 2, 4, 8, 12, 序列长度为 24 或 32; 数据覆盖零行、重复行、非单位、低秩耦合及后缀秩下降。80,140 条大模数查询用例中有 46,386 个 YES、33,754 个 NO, 并分别在追加时、历史版本和离线接口上核对。

性能样例使用固定种子 120926、 $n = 4000$, $d = 16$, $q = 8000$, 覆盖模 72、 $(10^9 + 7)^2$ 及上述 120 位 CRT 模数。每组抽查 83 个答案, 计时仅包围求解, 不含数据生成与 oracle 验证。一次运行不是统计学性能评估, 因此本文不据此声称最坏时间、最优性或相对正规形算法的加速比。原始时间和计数保存在 `artifacts/benchmarks.txt`。

工件复现。 仓库入口为 <https://github.com/Zi-Gao/ASTRA4OI/tree/main/results/composite-modulus-basis>。Lean 工具链固定为 4.32.0, Mathlib 固定为 v4.32.0 及锁定提交。在成果目录执行 `make test`、`make formal`、`make paper`, 分别复现程序测试、论文声明审计及中英文 PDF。编译和测试日志位于 `artifacts/`; 初始化下载与缓存步骤见使用文档。

9 讨论与结论

本方法将非单位主元引起的闭包需求组织为循环商群的有限幂层, 并利用保守时间戳同时维护所有后缀。正确性依赖首项唯一性、必要层不丢失及优先处理较新时间; 复杂度依赖每个新向量仅启动 k 条不分叉链。这三项条件共同给出任意合法输入上的一般结果, 而不是由大参数实验外推。

进一步的问题包括更精细的位成本、因数分解接口、系数见证恢复、修改或删除操作, 以及与其他动态模算法的系统比较。一般有限交换环的加法群归约保存在补充文档中, 其维度膨胀和形式化边界与本文整数模数主定理分开陈述。

参考文献

- [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [2] Ravindran Kannan and Achim Bachem. Polynomial algorithms for computing the Smith and Hermite normal forms of an integer matrix. *SIAM Journal on Computing*, 8(4):499–507, 1979.
- [3] Margreta Kuijper and Raquel Pinto. On minimality of convolutional ring encoders. <https://arxiv.org/abs/0801.3703>, 2009. arXiv:0801.3703, version 2.
- [4] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020.